Building a Business Context OS

that lets your LLM make better judgement calls

Claude Code Meetup · April 27, 2026 · Anna Ursin

Who am I?

Anna Ursin

- Fractional GTM consultant for B2B SaaS/tech (€1-5M ARR)
- I design and manage my clients' whole GTM engines: analysis, decisions, partners, channels – my work is dependent on good quality context, both for myself and my AI workflows
- Not "technical", but 95% of my work days are inside the terminal — not tools or UI

What this talk is – and isn't

- Design principles for a Context OS that supports better judgment calls
  - The codified knowledge
  - The architecture
  - The thinking tools
- IS NOT *(separate topics – check my blog for more on these)*
  - how to keep it up to date
  - how to make it collaborative

> ***"I don't use AI to replace thinking. I use it to never think the same thing twice."***

The problem isn't a LACK of context – but the form of it

Everyone already knows they should have well-structured context files

The actual problem: adding a few subheadings and more facts doesn't mean your context is designed for LLM judgment calls

Most people think facts = context. They're not the same thing – especially not for LLMs.

What's a judgment call?

- The small decisions your LLM makes when processing information

Facts without codified context

Defaults to plausible-sounding generic answers = the exact AI slop everybody hates

Facts with well-codified context

Applies YOUR business reality and its nuances to the call

Three prerequisites

1

Codified Knowledge

How you encode what you know

2

Architecture

How the LLM finds and reads it

3

Thinking Tools

How the LLM reasons about it

PART 1 — CODIFIED KNOWLEDGE

Codified knowledge: What goes in

- **Strategy:** ICP, positioning, financials, growth model, KPIs
- **Operations:** processes, tools, team, budget, partners
- **People:** who decides / executes what, expertise, preferences
- **Product:** features, roadmap, gaps, docs
- **History:** decisions, performance, what was tried, what worked
- **Tooling:** CLI, APIs, MCPs, scripts, skills, hooks (+ other tools not available to Claude Code)

The list itself is obvious; the non-obvious part is *how you codify it.*

Bad context vs. Good context

Bad: Facts dump

```
We're a SaaS for professional
services. ARR around 2M. 150
customers in the Nordics.
Growing 25% per year. Compete
with Harvest, Forecast,
Productive. ICP is firms 20-20
employees. Pain: don't know
which projects are profitable.
Planning DACH expansion.
Maria CEO, Erik sales, Lisa
marketing.
```

Good: Codified for judgment

```
## ARR: €1,942K (Q4 2025)
NRR: 108% | Churn: 8.4%
Customers: 147

## ICP — VALIDATED:
A-Invest: IT consult 62% win
B-Selective: Engineering 35%
Anti-ICP: 'just timesheets'

## Decisions:
DACH paused — no validated ICP
```

Fact dump vs. Codified context

Fact dump:

- No context, just facts — LLM reads everything as equally important or makes wild guesses
- No hierarchy — LLM reads it as a blob, can't tell what's important, misses and skips things
- No source attribution — LLM doesn't know how to verify if in doubt

Codified context:

- Structured, hierarchical, most important data points only ("will this affect our decision-making?")
- Every fact and number sourced and dated
- Patterns and validation status made explicit and actionable
- Designed for judgment, not just retrieval: LLM can find information if needed, qualify, flag anti-patterns, catch misleading metrics

Design principles – From fact dump to codified context

Principle	Fact dump
Source every fact	*"3M ARR"*
Codify the context, not just the data	*"NRR 115.6%"*
Validate claims against data	*"ICP is mid-marke finance"*
Separate validated from hypothesized	*"ICP: planning consultancies an cities"*
Make information actionable	*"We sell to financ*

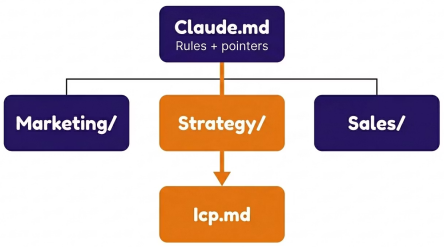
PART 2 – THE ARCHITECTURE

Requirements

- **Findability** — can the LLM locate the right file without searching the whole tree?
- **Readability** — can the LLM understand what it's looking at?
- **Version history** — can the LLM trust what it's reading, and roll back what it broke?
- **Instruction-following** — does the LLM actually follow the rules you set?

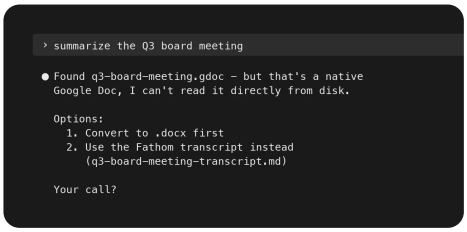
Findability

- Location – human-friendly enough that humans and LLMs can work on the same files.
  - Obsidian, Notion, GitHub, local folders
- Hierarchy
  - Uniform folder structures and naming conventions
- Search indexing
  - Pointer files (one source of truth per fact) + disciplined hierarchy



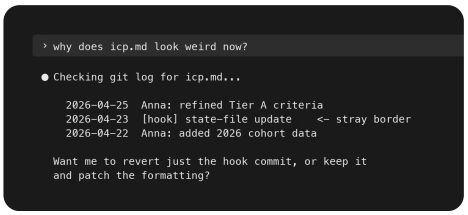
Readability

- File headers, usage instructions, structured formats over free-form prose, format consistency
- Nothing new here, but a reminder that format is a design decision and shapes how the AI interacts with data
- Readable → LLM has no excuse to skip
  - Markdown for prose / knowledge to reason about, CSV/SQL for tabular data, JSON for state data, etc.
- Unreadable → token-heavy, slow, compliance suffers (LLM starts skipping even when rules say to read)
  - Native Google files, image-only or scanned PDFs (no text layer), slides with content rendered as image, audio and video without transcripts…



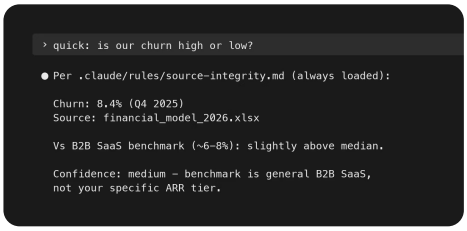
Version history

- Why LLMs need version history:
  - Roll back when the LLM breaks formatting or deletes something
  - Audit trail — see what changed, when, and whether human or LLM did it
  - Distinguish current truth from historical state — same fact at different points in time
- Even if working from human-first locations like Google Drive, consider using Git for the most important parts of the knowledge layer — not just code



Instruction compliance

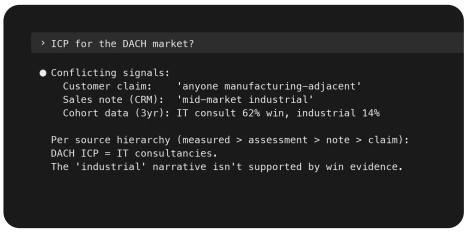
- You need LLMs to follow your instructions, but LLMs have a working instruction limit (~150 for best models). If you stuff every rule into the always-loaded layer, quality degrades fast.
- My preferences:
  - **Hooks** – Enforced compliance (e.g., read fully instead of skimming first paragraphs, ask before editing important files or using paid tools, etc.)
  - **Rules** (behavioral) — always loaded, but easily skipped → keep minimal (Is this relevant to 80%+ of sessions? No → reference file with pointer)
  - **Reference** (task-specific) — loaded on demand



PART 3 — THE THINKING TOOLS

Principle: strict trust hierarchy

- **File hierarchy**: e.g., constitution > context > memory > project logs
- **Data sources**: e.g., measured live data > own assessment > own notes > automatic notes > customer claims
- **Expertise**: e.g., domain expert > you > AI's general knowledge > web search
- **1st-order sourcing**: trace claims to original source
- **Analytical discipline**: bias patterns with corrective actions
- **Confidence marking**: e.g., fact (source?) / inference (what happened?) / hypothesis / needs investigation
- **Version authority**: working docs (latest wins) vs. research (newer ≠ supersede, just different)



PART 4 – WHAT IT ENABLES

Example 1: Sales call intelligence (500+ calls and growing)

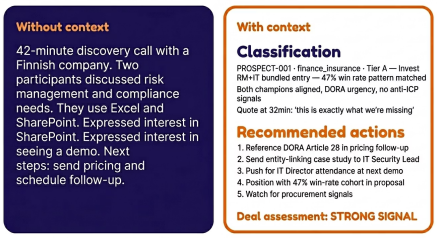
Input: messy auto-transcribed calls, sometimes no speaker labels, very little metadata

Process:

Codify → Enrich (internal + external sources) → Index → Analyze

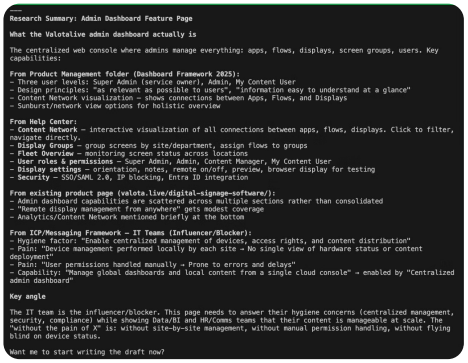
Outputs:

- Queryable index —> Pattern recognition (leadership, marketing)
- Codified per-call data



Example 2: Agentic content workflows

- Create any content (e.g., high-intent web pages, SEO pages, outbound email or ad copy, briefs, battlecards…) with minimal human interaction – ~20 min (5 mins of human work)
- Research — agent pulls from the codified knowledge base and live data (according to contextual rules)
- Drafting and revisions (text and images) – with built-in quality control loops until good enough
- (+ publishing, performance monitoring, next steps…)



```
---
Research Summary: Admin Dashboard Feature Page

What the Valotalive admin dashboard actually is

The centralized web console where admins manage everything: apps, flows, displays, screen groups, users. Key
capabilities:

From Product Management folder (Dashboard Framework 2025):
- Three user levels: Super Admin (service owner), Admin, My Content User
- Design principles: "as relevant as possible to users", "information easy to understand at a glance"
- Content Network visualization – shows connections between Apps, Flows, and Displays
- Sunburst/network view options for holistic overview

From Help Center:
- Content Network – interactive visualization of all connections between apps, flows, displays. Click to filter,
navigate directly.
- Display Groups – group screens by site/department, assign flows to groups
- Fleet Overview – monitoring screen status across locations
- User roles & permissions – Super Admin, Admin, Content Manager, My Content User
- Display settings – orientation, notes, remote on/off, preview, browser display for testing
- Security – SSO/SAML 2.0, IP blocking, Entra ID integration

From existing product page (valota.live/digital-signage-software/):
- Admin dashboard capabilities are scattered across multiple sections rather than consolidated
- "Remote display management from anywhere" gets modest coverage
- Analytics/Content Network mentioned briefly at the bottom

From ICP/Messaging Framework – IT Teams (Influencer/Blocker):
- Hygiene factor: "Enable centralized management of devices, access rights, and content distribution"
- Pain: "Device management performed locally by each site → No single view of hardware status or content
deployment"
- Pain: "User permissions handled manually → Prone to errors and delays"
- Capability: "Manage global dashboards and local content from a single cloud console" → enabled by "Centralized
admin dashboard"

Key angle

The IT team is the influencer/blocker. This page needs to answer their hygiene concerns (centralized management,
security, compliance) while showing Data/BI and HR/Comms teams that their content is manageable at scale. The
"without the pain of X" is: without site-by-site management, without manual permission handling, without flying
blind on device status.

Want me to start writing the draft now?
```